

Processing Time Prediction Model

Project Context & Learning Log

Organization: TRUMPF

Project type: Personal learning project (machine learning)

Primary goal: Learn ML properly by building a real, working processing-time predictor

Secondary goal: Explore whether this can become a sales-support tool later

Last updated: September 3, 2026

1. WHY THIS PROJECT EXISTS

Two goals, in this order:

1. **Learning goal (primary).** Understand machine learning end to end — how to look at data, how to pick an algorithm, how to read the metrics, and how to know when a model is lying to me.
2. **Business goal (secondary).** Predict how long a part takes to cut on a TRUMPF machine, so estimates stop depending on an experienced engineer's gut feeling.

Everything in this document serves goal 1 first. Goal 2 is the motivation that keeps the learning real.

2. THE PROBLEM

When a customer sends part files, someone has to answer: *how many seconds will this part take on this machine?*

Today that answer is:

- manual and slow
- dependent on one or two experienced people
- inconsistent between estimators

Target: predict `Processing_Time_Seconds` from part geometry, material, and program/nesting data.

3. DATA (what I actually have)

Source file: 7030 12kW.xlsx — one machine, TruLaser 7030 12kW

Raw: 9,587 rows × 34 columns

After `dropna()` + `drop_duplicates()`: 7,144 rows × 33 features + 1 target

The data was built by my own XML extractor (`xml_deep_scanner_v3.py`) which walks TRUMPF production XML files, converts every inch value to mm, and writes the unit into the column name.

The 33 features, grouped by where they come from

A. Part geometry (comes from the part itself)

Feature	Notes
Length_mm	bounding box
Width_mm	bounding box
Thickness_mm	part thickness
Area_mm2	part area
Weight_kg	mass
Aspect_Ratio	Length / Width
Part_Perimeter_mm	outer contour length
CutVolume_mm3	volume of removed material
Weight_per_Area_kg_per_m2	derived density measure

B. Cutting work (the strongest predictors)

Feature	Notes
CuttingLengthPart_mm	actual laser travel path — <i>not</i> the same as Length/Width
Piercing_Points	number of pierces on this part
TotalPiercingPointsInProgram	pierces across the whole program

CuttingLengthSheet_mm	total cutting on the sheet
PiercingDensity_per_m2	pierces per area
CuttingDensity_mm_per_m2	cutting length per area

C. Material & machine setup

Feature	Notes
Material_Group	Steel / Stainless / Aluminium (encoded)
RawMaterialThickness_mm	sheet thickness — the key to the speed table
Gas	assist gas (N2 / O2)
Kerf_mm	cut width
LensFocalLength_mm	optics setting
NumTechnoTables	how many techno tables the program uses

D. Program / nesting context

Feature	Notes
TotalQuantityInJob , TotalDesiredQuantity	order quantities
TotalNoOfRuns	production runs
NumPartTypesInProgram	part variety on the sheet
TotalQuantityOnSheets , NumPartsNested	nesting counts
TotalAreaOnSheet_mm2 , NestingEfficiency_Pct	nesting quality
AvgPositionX_mm ,	where the cuts sit on the sheet

AvgPositionY_mm	
,	
MaxPositionX_mm	
,	
MaxPositionY_mm	

Key insight recorded here so I don't forget it: CuttingLengthPart_mm is not redundant with Length_mm and Width_mm. Two parts with identical outer dimensions can have wildly different cutting paths depending on internal geometry. Bounding box = how big. Cutting length = how much work.

4. WHAT I BUILT — AND THE MISTAKE THAT TAUGHT ME THE MOST

Attempt 1: Linear Regression (failed, and that was useful)

Metric	Result
Train R ²	0.8964
Test R ²	0.7176
MAE	9.60 s
MAPE	42.7 %
Train – Test gap	17.9 %

Why it failed: I chose the algorithm before looking at the data. Processing time versus piercing points is a *curve*, not a straight line — going from 1 to 10 holes adds a lot of time, going from 100 to 110 adds very little. Linear Regression can only draw one straight line, so it was structurally wrong for this data.

Attempt 2: Gradient Boosting (current model)

Metric	Train	Test
R ²	0.9934 (99.34 %)	0.9372 (93.72 %)
MAE	4.24 s	4.73 s
MAPE	19.68 %	20.28 %

Overfitting gap	—	5.63 % → acceptable
-----------------	---	---------------------

Improvement over Attempt 1: +22 percentage points of test R^2 , half the average error, and the overfitting gap dropped from 17.9 % to 5.6 %.

Same data. Different algorithm. That was the whole difference.

5. THE RULE I NOW FOLLOW (the real deliverable of this project)

```

Load data
↓
Clean data
↓
PLOT the target against the main features ← the step I skipped last time
↓
Read the shape of the scatter:
  straight line      → Linear Regression is fine
  curve, flattening → log / power law → tree-based model
  accelerating      → exponential
  random cloud       → no relationship, stop and get better features
↓
THEN choose the algorithm
↓
Train, evaluate, check the train-test gap

```

Overfitting, in one line

Overfitting is memorising the practice exam instead of learning the subject: high train score, low test score.

Train – Test R^2 gap	Verdict
< 5 %	excellent
5–10 %	normal and acceptable ← current model at 5.63 %
> 10 %	investigate

Algorithm cheat sheet

Algorithm	Handles curves	Test R ² here	Use when
Linear Regression	no	0.7176	relationship is genuinely straight, need explainability
Random Forest	yes	~0.94	non-linear, want robustness with light tuning
Gradient Boosting	yes	0.9372	best accuracy with few parameters, ships with sklearn
XGBoost	yes	~same	huge datasets, competitions, squeezing the last fraction

6. THE PHYSICS BASELINE (why the model works)

Cutting Speeds_TruLaser 7030 (L26).pdf gives, for each material + thickness + gas combination:

- cutting speed (inch/min)
- pierce time (s)
- nozzle, gas pressure, gas consumption

That means a large part of processing time is simple physics:

```
processing_time ≈ (CuttingLengthPart_mm / cutting_speed)
+ (Piercing_Points × pierce_time)
+ overhead
```

Example — Steel, N2, ~1.27 mm: table speed ≈ 2,362 inch/min ≈ 60,000 mm/min, pierce ≈ 0.02 s.

What the ML model adds: the `overhead` term. Positioning moves, techno-table switches, nesting layout, acceleration and deceleration, quantity effects. That is exactly the part nobody can compute by hand — and it is why a physics formula alone is not enough.

Consequence for the future tool: physics gives the floor, the model corrects it. Both are needed.

7. CURRENT STATUS

Item	Status
XML extractor (xml_deep_scanner_v3.py)	working — 108 columns, all units in mm/kg, never moves or deletes source files
Training dataset (7030_12kW.xlsx)	working — 7,144 clean rows
EDA plots (histogram + scatter)	done — confirmed the curve
Gradient Boosting model	working — Test R² 93.72 %, MAE 4.73 s
Saved model (processing_time_model.pkl)	done via pickle
Prediction on a brand-new part	not done yet — needs all 33 features supplied
Feature extraction from STP	not started

Scope note: the model is trained on ONE machine (7030 12kW). It is valid for that machine only.

8. FUTURE POSSIBILITY — FROM LEARNING PROJECT TO TOOL

This is the "what could this become" section. It stays a *possibility*, not a commitment. Each phase must also teach me something new, otherwise it is not worth doing in this project.

Phase 0 — Prove the model on one unseen part (*next step, days*)

Learning goal: understand serialisation and the input contract of a trained model.

- Save the trained model with `pickle.dump()`

- Load it back with `pickle.load()`
- Hand-build a single-row DataFrame with all 33 features in the **same order** as training
- Predict, and compare against the actual known time
- Write down what happens when a feature is missing or in the wrong order

Why this matters: a model that cannot be reloaded and fed correctly is not a model, it is a notebook cell.

Phase 1 — Manual-entry predictor (1–2 weeks)

Learning goal: turn a model into something a second person can operate.

- Small script or form: user types the features they know
- Sensible defaults for machine constants (`Kerf_mm`, `LensFocalLength_mm`, `Gas`, `NumTechnoTables`)
- Physics fallback from the cutting-speed table when the model has no input
- Output: predicted seconds + the ± 4.73 s expected error, stated plainly
- Log every prediction so accuracy can be reviewed later

Phase 2 — Geometry from STP (*needs the engineer meeting*)

Learning goal: feature engineering from raw CAD instead of pre-made columns.

Directly extractable from an STP file:

`Length_mm`, `Width_mm`, `Thickness_mm`, `Area_mm2`, `Weight_kg` (volume $\times$ density),
`Part_Perimeter_mm`, `Aspect_Ratio`, `CutVolume_mm3`, `Weight_per_Area_kg_per_m2`

Needs a rule or a real algorithm:

- `Piercing_Points` — count closed internal contours
- `CuttingLengthPart_mm` — sum all cut contour lengths (outer + inner)

Needs user input or a default:

- `Material_Group`, `RawMaterialThickness_mm`, `Gas`, quantities

Needs nesting logic (hardest):

- `NumPartsNested`, `NestingEfficiency_Pct`, `TotalAreaOnSheet_mm2`, `CuttingLengthSheet_mm`, positions
- Test extraction on 2178970-00-A.stp
- Compare extracted values against the same part's XML values — this is the accuracy test that decides whether Phase 2 is viable

Phase 3 — Drop-a-file tool (*only if Phase 2 works*)

Learning goal: deployment and error handling.

- Folder or upload zone → predictions in Excel
- Flag parts that fall outside the training range instead of guessing silently
- Show which inputs were extracted, which were assumed

Phase 4 — Multi-machine (*long term*)

Learning goal: generalisation and model selection across segments.

- Repeat the pipeline for other machines / power classes
- Decide: one model with Machine as a feature, or one model per machine
- Validate on a completely held-out project, not just a random test split

Ideas worth trying purely for learning

- Feature importance — which of the 33 actually carry the signal
- Drop the weakest features and see whether test R^2 holds
- Predict $\log(\text{Processing_Time})$ instead of raw seconds and compare
- Hyperparameter tuning with GridSearchCV
- Compare against XGBoost on identical splits
- Cross-validation instead of a single 80/20 split

9. QUESTIONS FOR THE MACHINE ENGINEER — TUESDAY

Feature extraction

1. Can Piercing_Points be counted reliably from CAD, or is it decided during programming?
2. What exactly defines CuttingLengthPart_mm — the programmed laser path, or the geometric contour sum? How far apart are they?
3. Does an STP file carry material and sheet thickness, or must those always be entered?

Nesting

4. Is there access to nesting outputs (NumPartsNested, NestingEfficiency_Pct, CuttingLengthSheet_mm)?
5. How much does nesting actually move the processing time for a single part?

Machine constants

6. Are `Kerf_mm`, `LensFocalLength_mm` and `Gas` machine defaults that can be hard-coded per machine?

Prioritisation

7. If only five features were available, which five would matter most?

8. What sits inside the ~6 % of variance the model cannot explain — setup, operator, tool wear, something else?

10. LIMITATIONS (stated honestly)

What the model can do

- Predict processing time for parts similar to the 7,144 it was trained on
- On the TruLaser 7030 12kW only
- With roughly ± 4.73 s average error, ~20 % MAPE

What it cannot do

- Work on other machines without retraining
- Handle parts far outside the training range (extremely large, extremely dense)
- Explain a single prediction the way a linear equation could
- Replace an engineer's judgement — it produces an estimate, not a quote

Known risks

Risk	Mitigation
Feature values from STP differ from XML values	validate on a part where both exist before trusting Phase 2
Model used for a different machine	hard-code the machine name in the tool and refuse mismatches
MAPE ~20 % on very short parts	report absolute error alongside percentage error
Silent extrapolation	range-check inputs and warn

11. FILES IN THIS PROJECT

File	Role

<code>xml_deep_scanner_v3.py</code>	XML → Excel feature extractor, all units normalised to mm/kg
<code>7030_12kW.xlsx</code>	training dataset, 7,144 clean rows × 33 features
<code>train_and_predict_processing_time.py</code>	training script
<code>processing_time_model.pkl</code>	saved Gradient Boosting model
<code>Result.xlsx</code>	actual vs predicted vs error
<code>Cutting Speeds_TruLaser 7030 (L26).pdf</code>	physics reference — speeds and pierce times
<code>Feature_Importance_Guide.md</code>	notes on judging which features matter
<code>2178970-00-A.stp</code>	test part for Phase 2 extraction

Version: 2.0 — rewritten to reflect the Gradient Boosting model and the learning path that produced it